

A3 Delta Lake

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

- Track A: Databricks pipeline engineering.....2**
 - A3 Delta Lake 3
 - A3-L1 Idempotent MERGE, and why you deduplicate the source first..... 3
 - Track A best-practice checklist 10
- Appendix: solutions and answer keys11**

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A3 Delta Lake

Roadmap: Months 16 to 18: MLOps and deployment; Current work. MERGE, idempotent writes, schema enforcement and evolution, time travel, maintenance.

A3-L1 Idempotent MERGE, and why you deduplicate the source first

Concept

A pipeline is **idempotent** when running it twice with the same input leaves the table in the same state as running it once. Retries, reruns and redeliveries happen every week in production, so idempotence is not optional.

An append is not idempotent. Run `write.mode("append")` twice and every row lands twice. A Delta `MERGE` is idempotent when two conditions hold:

1. The `ON` condition covers every column of the table's grain. For a silver table with one row per respondent per wave, that means `respondent_id` **and** `wave_id`.
2. The source holds at most one row per key.

The second condition catches people out. When two source rows match the same target row, Delta cannot tell which one should win. It stops the `MERGE` with the error `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. When neither row matches a target row, both rows are inserted, and your grain is broken without any error.

So deduplicate the source before you merge. Rank the rows for each key by a version column, such as `delivery_version` or `ingestion_timestamp`, and keep the newest. Use a window with `row_number()`. Do not use `dropDuplicates()`: it keeps an arbitrary row per key, so a rerun keeps a different version with no warning.

The pattern for every silver load:

- Keep the newest source row per key.
- `MERGE` on the full grain.
- `whenMatchedUpdateAll()` for redelivered rows, `whenNotMatchedInsertAll()` for new ones.

Bronze stays append-only, because bronze records every delivery as received. Silver is where you resolve versions.

Why it matters

A supplier redelivers a wave with corrected weights. The file repeats 37 respondents, because the supplier's export joined two tables. The silver job uses the reference `upsert_silver`, which merges without deduplicating. The `MERGE` fails at 02:00 with

`DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Gold does not refresh, and the morning Power BI report shows last week's numbers.

The on-call engineer "fixes" it by switching the job to append. The next run succeeds. Silver now holds the wave twice. Reach percentages look normal, because both the numerator and the denominator doubled. Weighted population estimates are twice their true size for a month, until a reconciliation check against the weight total catches it.

Deduplicating on the full grain before the `MERGE` prevents both failures.

Worked example: weak code

```
1 from delta.tables import DeltaTable
2 from pyspark.sql import DataFrame, SparkSession
3
4 spark = SparkSession.builder.getOrCreate()
5
6
7 def upsert_silver(new_df: DataFrame, target_table: str, merge_keys: list)
  -> None:
8     """Merge new data into silver table."""
9     delta_table = DeltaTable.forName(spark, target_table)
10    merge_condition = " AND ".join(
11        [f"target.{k} = source.{k}" for k in merge_keys]
12    )
13    (
14        delta_table.alias("target")
15        .merge(new_df.alias("source"), merge_condition)
16        .whenMatchedUpdateAll()
17        .whenNotMatchedInsertAll()
18        .execute()
19    )
```

1. **Line 4.** A session created at import time. Tests cannot inject a local session, and importing the module has a side effect.
2. **Line 7.** `list` with no element type. mypy cannot check that callers pass column names.

3. **Line 15.** The source goes straight into the MERGE. One repeated key in a redelivery fails the whole job.

Worked example: strong code

```
1 from delta.tables import DeltaTable
2 from pyspark.sql import DataFrame, SparkSession, Window
3 from pyspark.sql import functions as F
4
5
6 def latest_per_key(df: DataFrame, keys: list[str], order_col: str) ->
    DataFrame:
7     """Keep the newest row for each key.
8
9     Args:
10         df: Source rows, possibly with repeated keys.
11         keys: Columns that identify one target row.
12         order_col: Column that ranks versions. Highest wins.
13
14     Returns:
15         One row per key.
16     """
17
18     window = Window.partitionBy(*keys).orderBy(F.col(order_col).desc())
19     return (
20         df.withColumn("_rank", F.row_number().over(window))
21         .filter(F.col("_rank") == 1)
22         .drop("_rank")
23     )
24
25 def upsert_silver(
26     spark: SparkSession,
27     source: DataFrame,
28     target_table: str,
29     keys: list[str],
30     order_col: str,
31 ) -> None:
32     """MERGE the newest source row per key into a silver table.
33
34     Running it twice with the same source leaves the table unchanged.
35     """
36     condition = " AND ".join(f"t.{k} = s.{k}" for k in keys)
37     (
38         DeltaTable.forName(spark, target_table).alias("t")
39         .merge(latest_per_key(source, keys, order_col).alias("s"),
40             condition)
41         .whenMatchedUpdateAll()
42         .whenNotMatchedInsertAll()
43         .execute()
```

1. **Line 3.** Import the module as `F`. Importing `sum` or `max` by name shadows the Python built-ins.
2. **Line 17.** One partition per key, newest version first. The result is the same on every run.
3. **Line 19.** `row_number()` gives exactly one row rank 1 per key, even when versions tie. `rank()` would keep ties.
4. **Line 26.** The session is a parameter. A test passes a local session, a job passes the cluster session.
5. **Line 39.** Deduplicate inside the MERGE call, so no caller can skip it.

The strong version is idempotent for any source: repeated keys, a retry or a full redelivery. The weak version works only while the supplier never repeats a row.

Recap card

- **When is a pipeline idempotent?** When running it twice with the same input leaves the table as one run would.
- **Two conditions for an idempotent Delta MERGE** The ON condition covers the full grain, and the source has at most one row per key.
- **What happens when two source rows match one target row?** The MERGE fails with `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`.
- **Why not drop Duplicates() before a MERGE?** It keeps an arbitrary row per key. Use `row_number()` over a window ordered by version, newest first.

Practice

Exercise 1. Multiple choice (level 1)

The silver table holds one row for respondent `R1` in wave `W1`. A redelivery holds **two** rows for `R1` in `W1`. You run a MERGE on `respondent_id` and `wave_id` with `whenMatchedUpdateAll()` and `whenNotMatchedInsertAll()`. What happens?

- a) The target row takes the values of the last source row.
- b) The MERGE fails with an error.
- c) Both source rows are inserted as new rows.
- d) The target row takes the values of the first source row.

Why?

- r1) Two source rows match one target row, and Delta cannot tell which should win.
- r2) Delta enforces a primary key on the target table.
- r3) The source DataFrame must be cached before a MERGE.

Exercise 2. Fill the gap (level 2)

Complete the chain so that redelivered rows update, new rows insert, and the operation runs.

```
1 (
2     DeltaTable.forName(spark,
3         "research_silver.nat_respondents").alias("t")
4     .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND
5         t.wave_id = s.wave_id")
6     ._____ (1) ()
7     ._____ (2) ()
8     ._____ (3) ()
9 )
```

Exercise 3. Predict the output (level 2)

`silver` starts empty. `source` holds 3 rows with 3 distinct (`respondent_id`, `wave_id`) keys. `load()` calls the strong `upsert_silver` from the worked example. What does the last line print?

```
1 load(source) # MERGE on respondent_id and wave_id
2 load(source) # the scheduler retries the same run
3 print(spark.table("silver").count())
```

Exercise 4. Spot the bug (level 3)

Silver has one row per respondent per wave. After wave `w2` loads, every wave `w1` row has disappeared. Tap the line with the bug, then fix the code.

```
1 def upsert_respondents(spark: SparkSession, source: DataFrame) -> None:
2     """MERGE one wave of respondents into silver. Grain: respondent per
3     wave."""
4     latest = latest_per_key(source, ["respondent_id", "wave_id"],
5         "delivery_version")
6     (
7         DeltaTable.forName(spark,
8             "research_silver.nat_respondents").alias("t")
9         .merge(latest.alias("s"), "t.respondent_id = s.respondent_id")
10        .whenMatchedUpdateAll()
11        .whenNotMatchedInsertAll()
12        .execute()
13    )
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Write the function (level 4)

Write `latest_per_key(df, keys, order_col)`. It returns one row per key: the row with the highest `order_col`. Use type hints and a docstring. The checker reads your code without running Spark.

Starter code:

```
1 from pyspark.sql import DataFrame, Window
2 from pyspark.sql import functions as F
3
4
5 def latest_per_key(df: DataFrame, keys: list[str], order_col: str) ->
    DataFrame:
6     ...
```

Exercise 6. Put in order (level 5)

Put the steps of an idempotent silver load in order.

- Write one record to the pipeline run log
- Rename and cast columns from the Variable Registry
- Validate the bronze delivery against its data contract
- MERGE into silver on `respondent_id` and `wave_id`
- Run the table health check on silver
- Keep the newest row per respondent and wave

Practical: Prove a MERGE is idempotent

Platform: Databricks Free Edition. Run this in Databricks Free Edition on serverless compute. It creates a schema called `learning_a3` in your current catalog, loads the same data twice, and shows that the row count does not change. It then repeats a key and shows the error.

Download the notebook `A3-L1.ipynb` from the app.

Sources

- MERGE fails with `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE` when several source rows match one target row. (verified: `practice/tests/test_a3_l1_merge.py` reproduces the error on delta-spark 4.0.1. Docs: <https://docs.databricks.com/aws/en/sql/language-manual/delta-merge-into>)
- `DeltaTable.forName`, `alias`, `merge`, `whenMatchedUpdateAll`, `whenNotMatchedInsertAll`, `whenNotMatchedBySourceDelete` and `execute`

exist in delta-spark 4.0. (verified: `practice/tests/test_a3_l1_merge.py` checks the methods on delta-spark 4.0.1.)

- Free Edition users can create schemas in the workspace catalog, whose name matches the workspace. (search-only: <https://docs.databricks.com/aws/en/catalogs/default>)

Track A best-practice checklist

- ☐ Every function has type hints, a docstring and one job.
- ☐ No SparkSession is created at import time. Functions receive spark as a parameter.
- ☐ Functions are imported as F: `from pyspark.sql import functions as F`.
- ☐ Table names, column maps and thresholds come from config or registry tables.
- ☐ Every table states its grain in a comment or table property.
- ☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.
- ☐ Every run writes one record to the run log table with an explicit schema.
- ☐ CRITICAL checks raise. WARN checks log and continue.
- ☐ Every division has a guard for zero and null.
- ☐ Counts come from one agg pass, not repeated count() calls.
- ☐ Incremental loads compare source and target on the same keys.
- ☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.
- ☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A3-L1 Idempotent MERGE, and why you deduplicate the source first

A3-L1 exercise 1

Hints. (1) Count how many source rows match the one target row. (2) Ask yourself which of the two source rows Delta should use for the update. (3) Delta raises `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE` in this case.

Answer. b) The MERGE fails with an error. Because: r1) Two source rows match one target row, and Delta cannot tell which should win.

Delta requires at most one source row per matched target row. With two, the update is ambiguous, so the `MERGE` fails with

`DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Deduplicate the source first.

Common wrong answers.

- a: Delta has no notion of 'last' in an unordered DataFrame. Row order in Spark is not guaranteed, so Delta refuses to guess.
- d: Spark gives no row order without an explicit sort, so 'first' has no meaning. Delta refuses to guess.
- c: Both rows match the existing target row, so neither goes to the insert clause. Duplicate inserts happen only when no target row matches.
- b|r2: Right outcome, wrong reason. Delta does not enforce primary keys. The error comes from the ambiguous match, not from a key constraint.
- b|r3: Right outcome, wrong reason. Caching has nothing to do with it. The error comes from two source rows matching one target row.

A3-L1 exercise 2

Hints. (1) Two clauses and one action. (2) The clause names start with `whenMatched` and `whenNotMatched`, and both copy every column. (3) The copy-every-column versions end in `ALL`. The action that runs a MERGE is not `collect`.

Answer.

```
1 (
2   DeltaTable.forName(spark,
3     "research_silver.nat_respondents").alias("t")
4   .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND
5     t.wave_id = s.wave_id")
6   .whenMatchedUpdateAll()
7   .whenNotMatchedInsertAll()
8   .execute()
9 )
```

`whenMatchedUpdateAll()` copies every source column onto the matched target row. `whenNotMatchedInsertAll()` inserts rows with no match. `execute()` runs the MERGE.

Common wrong answers.

- `g3 = collect: collect()` is a DataFrame action. A MERGE builder runs with `execute()`.
- `g1 = whenMatchedUpdate: whenMatchedUpdate()` needs a `set=` mapping of columns. With no arguments you want `whenMatchedUpdateAll()`.
- `g2 = whenNotMatchedBySourceDelete: whenNotMatchedBySourceDelete()` deletes target rows that are missing from the source. With a one-wave source it would delete every other wave.

A3-L1 exercise 3

Hints. (1) Work through the first run: which clause fires for each row? (2) On the second run, does any source row lack a match? (3) Matched rows update in place. Updates do not add rows.

Answer.

```
1 3
```

Run one inserts 3 rows. Run two matches all 3 and updates them with identical values. The count stays 3. That is idempotence.

Common wrong answers.

- `6`: 6 is what an append would give. On the second run every source row matches an existing target row, so the MERGE updates instead of inserting.
- `0`: The first run inserts all 3 rows, because none of them matches an empty table.

A3-L1 exercise 4

Hints. (1) The grain is respondent per wave. Which columns identify one row? (2) Compare the deduplication keys with the MERGE condition. (3) A W2 row for respondent R1 matches the W1 row for R1, and overwrites it.

Answer. The bug is on line 6.

Fixed version:

```
1 def upsert_respondents(spark: SparkSession, source: DataFrame) -> None:
2     """MERGE one wave of respondents into silver. Grain: respondent per
   wave."""
3     latest = latest_per_key(source, ["respondent_id", "wave_id"],
   "delivery_version")
4     (
5         DeltaTable.forName(spark,
6         "research_silver.nat_respondents").alias("t")
7         .merge(latest.alias("s"), "t.respondent_id = s.respondent_id AND
8         t.wave_id = s.wave_id")
9         .whenMatchedUpdateAll()
10        .whenNotMatchedInsertAll()
11        .execute()
12    )
```

The condition matches on `respondent_id` alone. A wave W2 row for a respondent matches that respondent's W1 row and overwrites it, including `wave_id`. Match on the full grain: `t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id`.

Common wrong answers.

- Line 3: Line 3 deduplicates on both key columns, which matches the grain. Look at what the MERGE matches on.
- Line 7: `whenMatchedUpdateAll()` is right for a redelivery. The problem is which rows count as a match.

A3-L1 exercise 5

Hints. (1) You need a window, a rank and a filter. (2)

`Window.partitionBy(*keys).orderBy(F.col(order_col).desc())` sorts each key's rows newest first. (3) Add `F.row_number().over(window)` as a column, keep rank 1, then drop the helper column.

Answer.

```
1 from pyspark.sql import DataFrame, Window
2 from pyspark.sql import functions as F
3
4
5 def latest_per_key(df: DataFrame, keys: list[str], order_col: str) ->
   DataFrame:
```

```

6     """Keep the row with the highest order_col for each key.
7
8     Args:
9         df: Source rows, possibly with repeated keys.
10        keys: Columns that identify one target row.
11        order_col: Column that ranks versions. Highest wins.
12
13    Returns:
14        One row per key.
15    """
16    window = Window.partitionBy(*keys).orderBy(F.col(order_col).desc())
17    return (
18        df.withColumn("_rank", F.row_number().over(window))
19        .filter(F.col("_rank") == 1)
20        .drop("_rank")
21    )

```

A window partitioned by the keys and ordered by `order_col` descending puts the newest row first in each key. `row_number()` numbers them 1, 2, 3. Keep 1, drop the helper column.

A3-L1 exercise 6

Hints. (1) Nothing unchecked should reach silver. (2) The MERGE needs a source with one row per key. (3) The run log records the outcome of everything before it.

Answer.

1. Validate the bronze delivery against its data contract
2. Rename and cast columns from the Variable Registry
3. Keep the newest row per respondent and wave
4. MERGE into silver on `respondent_id` and `wave_id`
5. Run the table health check on silver
6. Write one record to the pipeline run log

Check the delivery, map it to canonical names, deduplicate, merge, confirm the table is healthy, then log the run with its final status.

Common wrong answers.

- Deduplicating after the MERGE is too late. The MERGE has already failed, or has already inserted the repeated rows.
- The contract describes the delivery as received. Check it before you rename, or the column names in the contract no longer match.
- Log last, so the run record includes the health check result and the final status.